

Análisis del Comportamiento de E-Converse bajo Diferentes Escenarios de Despliegue

Proyecto Final

Ingeniería de Software III

Universidad Industrial de Santander

Equipo:

Juan Sebastián Otero — 2220053

Farid Camilo Rojas Vargas — 2220051

Juan David Pallares Pallares — 2220079

Mayo 2026

Índice

1. Introducción	2
2. Descripción de la Aplicación: E-Converse	2
2.1. Funcionalidades principales	2
2.2. Endpoints utilizados para las pruebas de carga	2
3. Estructura del Repositorio	3
4. Cómo Ejecutar el Proyecto	3
4.1. Requisitos previos	3
4.2. Opción A: Ejecución local sin contenedores (desarrollo)	3
4.3. Opción B: Docker Compose — Fase 1	4
4.3.1. Arquitectura del despliegue	4
4.3.2. Archivos de configuración	4
4.3.3. Pasos para desplegar	5
4.3.4. Problemas conocidos y soluciones	6
4.4. Opción C: Kubernetes — Fases 2, 3 y 4	6
4.5. Poblar la base de datos con 10K productos	7
5. Cómo Visualizar el Notebook	7
5.1. Opción 1: Jupyter Lab/Notebook local	7
5.2. Opción 2: VS Code con extensión Jupyter	7
5.3. Opción 3: Google Colab	7
5.4. Re-ejecutar todas las celdas	7
6. Cómo Reproducir las Pruebas de Carga	8
6.1. Fase 1 — Docker Compose: pruebas manuales con JMeter GUI	8
6.2. Fases 2-4 — Kubernetes: plan parametrizado y script automático	8
7. Resumen de Resultados	8
7.1. Fase 1 — Docker Compose (AWS EC2)	9
7.2. Fases 2-4 — Kubernetes K3s	9
7.3. Conclusiones principales	9
7.4. Recomendaciones	10
8. Enlaces y Recursos	10

1 Introducción

Este documento es el reporte de entrega del proyecto final del curso. Su propósito es servir como guía operativa: explica qué es la aplicación, cómo está organizado el repositorio, cómo levantar el sistema en cada escenario de despliegue (local, Docker Compose y Kubernetes), cómo reproducir las pruebas de rendimiento con JMeter y dónde está el análisis completo de los resultados.

El **análisis técnico detallado**, las **tablas** con todas las métricas y las **gráficas comparativas** se encuentran en el notebook Jupyter `Analisis_Rendimiento.ipynb`. Este PDF se enfoca en las instrucciones de ejecución y los enlaces necesarios.

Las cuatro fases del proyecto son:

1. **Fase 1 — Docker Compose:** despliegue en una única VM de AWS EC2 (18.116.30.127), pruebas con 10, 50 y 100 usuarios concurrentes sobre los endpoints de Login y Productos.
2. **Fase 2 — Kubernetes 1 nodo:** clúster K3s con solo el nodo master activo, variando réplicas del backend (1, 2, 3).
3. **Fase 3 — Kubernetes 2 nodos:** master + worker, variando réplicas (1, 2, 3).
4. **Fase 4 — Kubernetes 3 nodos:** clúster completo, variando réplicas (1, 2, 3).

2 Descripción de la Aplicación: E-Converse

E-Converse es una plataforma de comercio electrónico con backend Spring Boot, frontend React y persistencia en MongoDB Atlas.

Componente	Tecnología
Backend	Spring Boot 3.5.6 (Java 17), API REST, JWT
Frontend	React 19 + Vite 7, TailwindCSS, Zustand
Base de Datos	MongoDB Atlas (10,000+ productos)
Contenedorización	Docker multi-stage
Orquestación	Kubernetes (K3s v1.35.5)
Pruebas de Carga	Apache JMeter 5.6.3
Análisis	Jupyter Notebook (pandas, matplotlib, seaborn)

2.1 Funcionalidades principales

- Autenticación con JWT (login, registro, roles administrador/cliente).
- Panel administrativo con CRUDs de usuarios, roles, categorías, productos y pedidos.
- Catálogo público con filtros por categoría, precio y nombre.
- Carrito de compras persistente en MongoDB con índice único por usuario.
- Proceso de checkout con simulación de pasarela de pago.
- Historial de pedidos por usuario.

2.2 Endpoints utilizados para las pruebas de carga

Endpoint	Método	Tamaño resp.	Característica
POST /api/auth/login	POST	~900 bytes	Autenticación JWT (Fase 1)
GET /api/categorias/list	GET	~800 bytes	Endpoint ligero (Fases 2–4)
GET /api/productos/list	GET	~3.5 MB	Endpoint pesado (todas las fases)

3 Estructura del Repositorio

```
Proyecto-Spring-Boot-E-converse_vers2/
|-- backend/                                # API Spring Boot (codigo +
    Dockerfile)
|-- frontend-react/                        # SPA React + Vite (codigo + nginx +
    Dockerfile)
|-- docker-compose.yml                    # Despliegue Fase 1 (Docker Compose)
|-- k8s/
|   |-- backend-deployment.yaml
|   |-- frontend-deployment.yaml
|-- jmeter/
|   |-- econverse_load_test.jmx          # Plan JMeter parametrizado (Fases
    2-4)
|   |-- run_all_tests.sh                  # Script automatizacion 9 escenarios
    K8s
|   |-- run_1node_only.sh                 # Re-ejecucion solo Fase 2
|   |-- results/                          # 9 CSVs K8s (corrida final, 0%
    errores)
|   |-- results_old/                      # 10 CSVs (corrida inicial, 67%
    errores)
|-- scripts/
|   |-- populate_db.js                    # Insercion de 10K productos
|   |-- setup_roles_admin.js              # Configuracion inicial
|   |-- redeploy-backend.sh              # Re-deploy backend en K3s
|-- Analisis_Completo_EConverse_v2.ipynb  # Notebook completo (Fases
    1-4)
|-- REPORTE_PROYECTO.md                   # Diario de problemas y soluciones
|-- Reporte_Final.tex / .pdf              # Este documento
|-- README.md
```

4 Cómo Ejecutar el Proyecto

4.1 Requisitos previos

- Java 17+ y Maven (o usar el contenedor Docker que ya incluye Maven).
- Node.js 22+ y npm (para frontend dev; Vite 7 requiere Node 20.19+).
- Docker y Docker Compose v2.
- Acceso a un clúster Kubernetes (K3s, MicroK8s, Kind o cloud).
- Una URI de MongoDB Atlas con acceso desde la IP de la VM.

4.2 Opción A: Ejecución local sin contenedores (desarrollo)

```
cd backend
# Configurar la URI en src/main/resources/application.properties:
# spring.data.mongodb.uri=mongodb+srv://<user>:<pass>@<cluster>/
#   econverse
mvn spring-boot:run
# Backend disponible en http://localhost:8080
```

Listing 1: Backend Spring Boot

```
cd frontend-react
```

```
npm install
npm run dev -- --host      # --host expone la red local
# Frontend disponible en http://localhost:5174
```

Listing 2: Frontend React

4.3 Opción B: Docker Compose — Fase 1

Esta fase despliega la aplicación completa en una única VM de AWS EC2 (18.116.30.127) usando Docker Compose. El frontend corre en Nginx (puerto 8081) y el backend en Tomcat embebido (puerto 8080).

4.3.1 Arquitectura del despliegue

```
Internet
|
:8081 (Nginx / frontend)
|
/api/* --> proxy_pass http://backend:8080/
|
:8080 (Spring Boot / backend)
|
MongoDB Atlas (cloud)
```

4.3.2 Archivos de configuración

docker-compose.yml:

```
services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    restart: unless-stopped
    dns:
      - 8.8.8.8      # necesario para resolver DNS de MongoDB Atlas
      - 8.8.4.4
    environment:
      SPRING_DATA_MONGODB_URI: mongodb+srv://user:pass@cluster0.xxx.
mongodb.net/?appName=Cluster0
    ports:
      - "8080:8080"

  frontend:
    build:
      context: ./frontend-react
      dockerfile: Dockerfile
    args:
      VITE_API_URL: /api      # el proxy nginx reescribe /api/ ->
backend:8080/
    restart: unless-stopped
    ports:
      - "8081:80"
    depends_on:
      - backend
```

frontend-react/nginx.conf:

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;

    location /api/ {
        proxy_pass http://backend:8080/;    # barra final elimina /api
        del path
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

backend/Dockerfile (multi-stage):

```
FROM maven:3.9.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

frontend-react/Dockerfile (multi-stage):

```
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG VITE_API_URL=/api
ENV VITE_API_URL=${VITE_API_URL}
RUN npm run build

FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4.3.3 Pasos para desplegar

1. Liberar puertos si el backend ya corre de forma nativa:

```
sudo fuser -k 8080/tcp
sudo fuser -k 5173/tcp 5174/tcp
```

2. Configurar el Security Group de AWS (puertos a abrir en Inbound Rules):

Type	Port	Source
Custom TCP	8080	0.0.0.0/0
Custom TCP	8081	0.0.0.0/0
SSH	22	0.0.0.0/0

3. Configurar CORS en application.properties (dentro de backend/src/main/resources/):

```
spring.web.cors.allowed-origins=http://localhost:5173,http://localhost:5174,\
http://18.116.30.127:8081
spring.web.cors.allowed-methods=GET,POST,PUT,DELETE,OPTIONS
spring.web.cors.allowed-headers=*
spring.web.cors.allow-credentials=true
```

4. Levantar con Docker Compose:

```
cd ~/Proyecto-Spring-Boot-E-converse_vers2
docker compose up --build -d
```

5. Verificar que los contenedores estén corriendo:

```
docker compose ps
docker compose logs backend | tail -20
# Debe aparecer: Tomcat started on port 8080
```

6. Acceder desde el navegador:

- Frontend: <http://18.116.30.127:8081>
- Backend (API): <http://18.116.30.127:8080>

4.3.4 Problemas conocidos y soluciones

Problema	Solución
crypto.hash is not a function en Vite	Node.js 18 no soporta Vite 7. Actualizar a Node 22 con NVM: <code>nvm install 22 && nvm use 22</code>
Address already in use en puerto 8080	Matar proceso nativo: <code>sudo fuser -k 8080/tcp</code>
Failed looking up TXT record (MongoDB Atlas)	Agregar <code>dns: [8.8.8.8, 8.8.4.4]</code> al servicio backend en <code>docker-compose.yml</code>
Error CORS desde el navegador	Agregar <code>http://<IP>:8081</code> a <code>spring.web.cors.allowed-origins</code> y reconstruir el backend
XML Parsing Error al hacer login	El proxy nginx devuelve HTML en lugar de JSON. Usar <code>proxy_pass http://backend:8080/</code> con barra final (sin <code>rewrite</code>)
Imagen cacheada de Docker	Reconstruir sin caché: <code>docker compose build --no-cache frontend</code>

4.4 Opción C: Kubernetes — Fases 2, 3 y 4

1. Construir las imágenes localmente:

```
docker build -t e-converse-backend:latest ./backend
docker build -t e-converse-frontend:latest ./frontend-react
```

2. Distribuir las imágenes a todos los nodos del clúster:

```
docker save e-converse-backend:latest -o backend.tar
docker save e-converse-frontend:latest -o frontend.tar
# En cada nodo del cluster:
sudo k3s ctr images import backend.tar
sudo k3s ctr images import frontend.tar
```

3. Configurar la URI de MongoDB en k8s/backend-deployment.yaml.

4. Desplegar:

```
sudo k3s kubectl apply -f k8s/backend-deployment.yaml
sudo k3s kubectl apply -f k8s/frontend-deployment.yaml
sudo k3s kubectl get pods -o wide
# Aplicacion disponible en http://<IP-master>:30080
```

4.5 Poblar la base de datos con 10K productos

```
cd scripts
npm install
node setup_rolas_admin.js      # Roles y usuario admin
node populate_db.js           # Insercion de 10K productos mock
```

5 Cómo Visualizar el Notebook

El análisis completo de las cuatro fases se encuentra en `Analisis_Rendimiento.ipynb` en la rama `soft3_final`. Hay tres formas de abrirlo:

5.1 Opción 1: Jupyter Lab/Notebook local

```
pip install jupyter pandas matplotlib seaborn
jupyter notebook Analisis_Rendimiento.ipynb
```

5.2 Opción 2: VS Code con extensión Jupyter

Abrir el archivo `.ipynb` desde VS Code; la extensión Jupyter detectará el kernel de Python automáticamente.

5.3 Opción 3: Google Colab

Subir el notebook y la carpeta `jmeter/results/` a Google Drive, abrir con Colab y montar Drive. El notebook lee los CSVs de K8s desde la ruta relativa `jmeter/results/`. Los datos de Docker Compose están codificados directamente en el notebook.

5.4 Re-ejecutar todas las celdas

```
jupyter nbconvert --to notebook --execute --inplace \
  Analisis_Rendimiento.ipynb
```


6 Cómo Reproducir las Pruebas de Carga

6.1 Fase 1 — Docker Compose: pruebas manuales con JMeter GUI

Las pruebas de la Fase 1 se ejecutaron desde la máquina local usando la interfaz gráfica de JMeter 5.6.3. El flujo de cada hilo fue:

1. POST /api/auth/login con credenciales válidas.
2. JSON Extractor extrae el campo `token` de la respuesta.
3. GET /api/productos/list con cabecera `Authorization: Bearer ${token}`.

Escenario	Usuarios	Ramp-up	Iteraciones
Carga Baja	10	10 s	5
Carga Media	50	20 s	5
Carga Alta	100	30 s	5

Nota: para la prueba de 100 usuarios, deshabilitar el *View Results Tree* y aumentar la memoria de JMeter editando `bin/jmeter` o `bin/jmeter.bat`:

```
HEAP="-Xms2g -Xmx4g"
```

6.2 Fases 2–4 — Kubernetes: plan parametrizado y script automático

El archivo `jmeter/econverse_load_test.jmx` es un plan con parámetros configurables:

Parámetro	Flag	Valor por defecto
Servidor	-Jserver	10.6.101.163
Puerto	-Jport	30080
Hilos concurrentes	-Jthreads	20
Ramp-up	-Jrampup	20 s
Duración	-Jduration	60 s

Ejecutar una prueba puntual:

```
flatpak run org.apache.jmeter -n \
-t jmeter/econverse_load_test.jmx \
-l jmeter/results/mi_prueba.csv \
-Jserver=10.6.101.163 -Jport=30080 \
-Jthreads=20 -Jrampup=20 -Jduration=60
```

Ejecutar la suite completa (9 escenarios K8s, 40–50 min):

```
./jmeter/run_all_tests.sh
# Resultados en jmeter/results/k8s_*.csv
```

7 Resumen de Resultados

Para el detalle completo (gráficas comparativas, heatmaps, curvas de escalado) ver el notebook. A continuación el resumen ejecutivo de todas las fases.

7.1 Fase 1 — Docker Compose (AWS EC2)

Escenario	Endpoint	Muestras	Avg (ms)	Min (ms)	Max (ms)	Error %
Carga Baja (10u)	Login	50	351	225	673	0
Carga Baja (10u)	Productos	50	1,440	589	3,676	0
Carga Media (50u)	Login	235	710	207	20,551	0
Carga Media (50u)	Productos	193	9,523	589	54,051	0
Carga Alta (100u)	Login	383	657	206	20,551	0
Carga Alta (100u)	Productos	246	8,040	589	54,051	0

Observaciones Fase 1:

- 0 % de errores en todos los escenarios.
- El endpoint de productos se degradó un 561 % al pasar de 10 a 50 usuarios.
- El cuello de botella principal es la latencia hacia MongoDB Atlas con un único contenedor backend sin escalado horizontal.

7.2 Fases 2–4 — Kubernetes K3s

Escenario	Nodos	Réplicas	Throughput Cat (req/s)	Latencia Cat (ms)	Error %
1N-1R	1	1	1.37	398	0.0
1N-2R	1	2	1.18	593	0.0
1N-3R	1	3	0.73	1,076	0.0
2N-1R	2	1	1.83	396	0.0
2N-2R	2	2	2.15	318	0.0
2N-3R	2	3	2.60	309	0.0
3N-1R	3	1	1.77	356	0.0
3N-2R	3	2	4.27	294	0.0
3N-3R	3	3	3.27	289	0.0

Configuración óptima global: 3 nodos × 2 réplicas (4.27 req/s, 294 ms de latencia, 0 % errores).

7.3 Conclusiones principales

1. **Docker Compose es adecuado solo para cargas bajas.** Con 10 usuarios el tiempo de respuesta de productos es de 1,440 ms; con 50 usuarios sube a 9,523 ms (+561 %). No permite escalado horizontal.
2. **El escalado horizontal en Kubernetes funciona, pero tiene un techo definido por MongoDB Atlas.** Pasar de 1 a 2 réplicas en 3 nodos más que duplica el throughput (+141 %); una 3a réplica lo reduce (−23 %) por saturación del connection pool en Atlas (plan compartido).
3. **Concentrar carga en un solo nodo es contraproducente.** El escenario 1N-3R es el peor de todos: 0.73 req/s y 16,131 ms de latencia en productos.
4. **El tamaño del payload domina sobre la topología.** La latencia de productos (3.5 MB) es 10–15× la de categorías en *todos* los escenarios, independientemente del número de nodos o réplicas.
5. **Kubernetes mejora el rendimiento respecto a Docker Compose** en la configuración óptima: latencia de productos pasa de 1,440 ms (DC, 10u) a 1,958 ms (3N-2R, 20u) con mayor carga concurrente y 0 % errores.

7.4 Recomendaciones

Problema	Solución propuesta
Contención en MongoDB Atlas	Migrar a plan dedicado o implementar caché Redis (<code>@Cacheable</code>)
Payload sin paginar (3.5 MB)	Implementar paginación (<code>?page=N&size=20</code>) — mayor ROI
Cold start de pods (30–60 s)	Configurar <code>readinessProbe</code> con <code>initialDelaySeconds: 30</code>
Imágenes inconsistentes entre nodos	Usar registry privado en lugar de <code>imagePullPolicy: Never</code>
Docker Compose no escala	Migrar a Kubernetes para producción con carga variable

8 Enlaces y Recursos

- Repositorio principal:
https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse_vers2
- Rama Docker Compose (Fase 1):
https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse_vers2/tree/despliegue-docker
- Rama K8s (Fases 2–4):
https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse_vers2/tree/soft3_final
- Video:
<https://youtu.be/ndrHBd8x-dw>
- Notebook de análisis completo (en el repositorio):
`Analisis_Rendimiento.ipynb`
- Diario de problemas y soluciones:
`REPORTE_PROYECTO.md`
- Documentación Apache JMeter:
<https://jmeter.apache.org/usermanual/get-started.html>
- Documentación K3s:
<https://docs.k3s.io/>
- MongoDB Atlas:
<https://www.mongodb.com/atlas>